TÀI LIỆU KỸ THUẬT TOÀN DIỆN: ROBOT VẬN CHUYỂN VẬT TƯ TỰ HÀNH

ĐỀ TÀI: Thiết kế robot vận chuyển vật tư tự hành trong môi trường trong nhà sử dụng Raspberry Pi 4 và STM32

Sinh viên thực hiện: Chống Định Khang (23161277) – Phạm Trung Kiên (23161283)
Giảng viên hướng dẫn: PGS. TS. Phan Văn Ca
Khoa/Bộ môn: Bộ môn Kỹ thuật Máy tính và Truyền thông – Trường ĐH Sư phạm Kỹ thuật TP.HCM (HCMUTE)

PHẦN 1: Ý TƯỞNG CỐT LÕI & BÀI TOÁN KỸ THUẬT THỰC TẾ

1. Bối cảnh & Vấn đề giải quyết

- Trong các phòng thí nghiệm điện tử, xưởng lắp ráp bo mạch thử nghiệm (SMT) hoặc trung tâm R&D, việc luân chuyển các khay linh kiện, IC mẫu, bo mạch thử nghiệm và thiết bị đo diễn ra liên tục giữa các bàn làm việc. Việc di chuyển thủ công làm phân tán thời gian nghiên cứu của kỹ sư.
- Đề tài xây dựng một mô hình robot di động tự hành cỡ nhỏ (**Mini AMR Prototype**) vận chuyển khay linh kiện giữa các trạm cố định trong phòng:
- **Trạm 1:** Kho linh kiện & IC mẫu (Mốc ArUco ID 1)
- **Trạm 2:** Bàn đo kiểm & máy hiện sóng (Mốc ArUco ID 2)
- **Trạm 3:** Bàn nạp code & kiểm tra chức năng (Mốc ArUco ID 3)

2. Định hướng kỹ thuật thực chiến & Phạm vi hoạt động

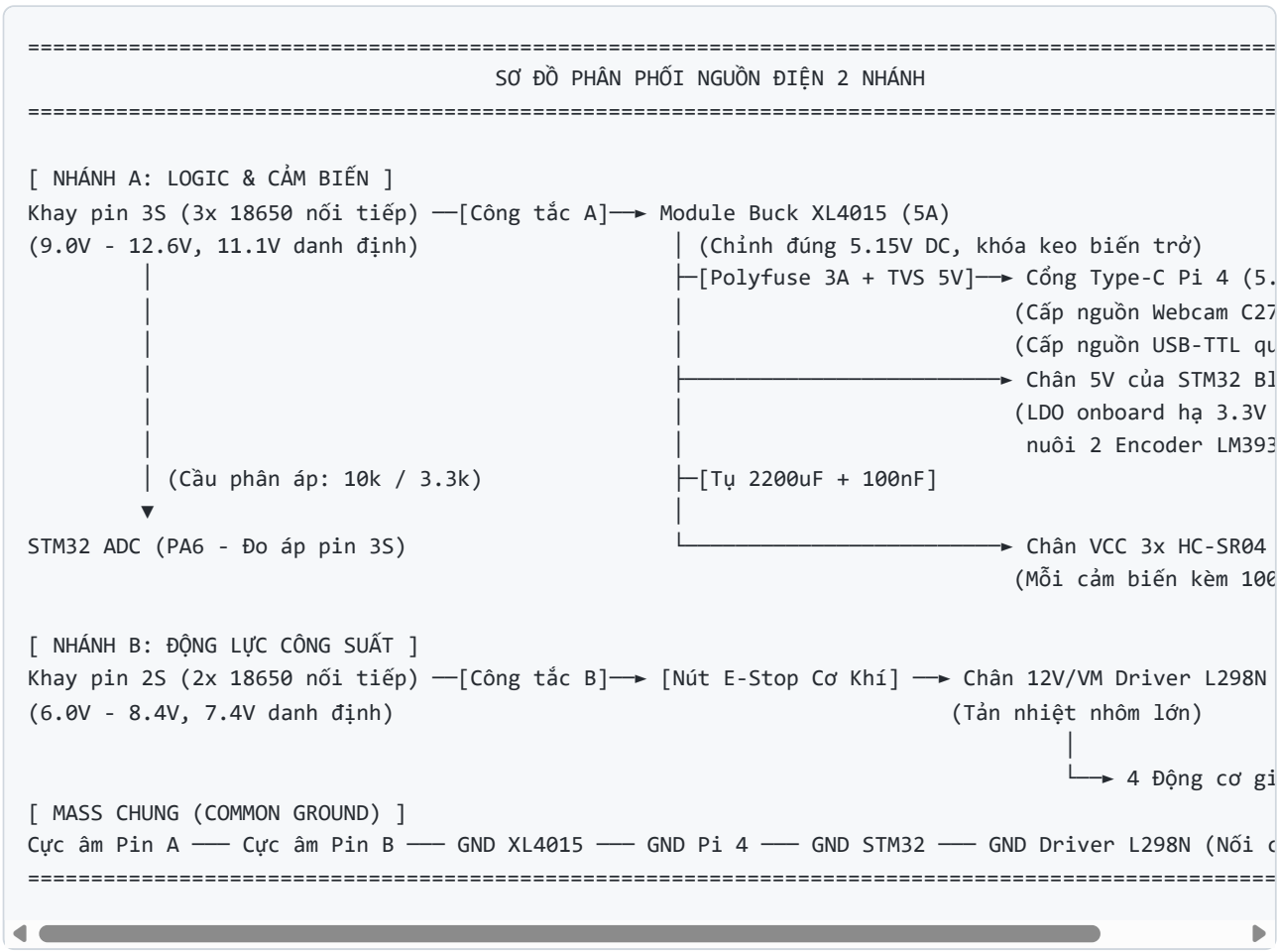
- **Không làm bản đồ 2D phức tạp (SLAM / LiDAR):** Robot giải quyết bài toán **Dẫn đường dựa trên mốc trực quan (Marker-based Navigation)** trong hành lang thông thoáng giữa các trạm (Line-of-Sight). Robot không cần xây dựng bản đồ toàn cục của cả tòa nhà, mà nhận lệnh từ giao diện Web, tự phát hiện mốc định danh của trạm đích và di chuyển tiếp cận trạm an toàn.
- **Cơ cấu Camera cố định vững chắc (Fixed Camera):** Sử dụng Webcam USB Logitech C270 (có sẵn) bắt vít cố định cứng vững vào khung xe. Robot quét tìm trạm bằng giải thuật xoay vi sai thân xe theo bước góc (*Step-Turn-and-Check*). Giải pháp này loại bỏ hoàn toàn cụm động cơ xoay servo lỏng lẻo, triệt tiêu rung lắc gây nhòe khung hình (*motion blur*) khi xe chạy trên sàn gạch và loại bỏ nguy cơ servo kéo sụt áp hệ thống.
- **Kiến trúc phân tầng Master – Slave tối ưu:** Phân tách rõ ràng giữa máy tính nhúng Raspberry Pi 4 (xử lý thị giác máy tính OpenCV, Web server Flask, quản lý trạng thái nhiệm vụ) và vi điều khiển thời gian thực STM32F103C8T6 (điều khiển xung PWM động cơ, đọc cảm biến đĩa quang encoder và ngắt an toàn phản xạ siêu âm dưới 20ms).
- **Phạm vi môi trường & Giới hạn hoạt động thực tế:**

- Hoạt động trong hành lang nội bộ có ánh sáng đồng đều ($\ge 150\text{ lux}$), sàn phẳng, không có gờ bậc hụt sàn lớn.
- Tầm nhìn thẳng trực tiếp (Line-of-Sight) giữa các trạm với cự ly làm việc hiệu quả từ 0.3m đến 2.2m.
- Mốc ArUco có kích thước cố định $18\text{cm} \times 18\text{cm}$, dán ở độ cao tâm mốc $25 - 30\text{cm}$ so với sàn.
- Vật cản được giả định có tiết diện phẳng/thẳng đứng đủ để phản xạ sóng siêu âm (tránh các mặt xiên $> 45^\circ$ hoặc rèm vải hấp thụ âm).

PHẦN 2: THIẾT KẾ PHẦN CỨNG & HỆ THỐNG ĐIỆN CHỐNG BROWNOUT

1. Phân tích Nguồn điện 2 Nhánh Độc lập & Giải pháp Chống Brownout

Một trong những rủi ro kỹ thuật lớn nhất của các hệ thống robot nhúng sử dụng Raspberry Pi 4 là hiện tượng **sụt áp đột ngột (Brownout)** dẫn đến treo hệ điều hành hoặc hỏng phân vùng thẻ nhớ SD khi CPU boost xung hoặc động cơ khởi động. Hệ thống khắc phục triệt để bằng thiết kế nguồn điện 2 nhánh cách ly hoàn toàn, vẽ rõ ràng đường cấp nguồn cho từng khối:



A. Phân tích Điện áp Dropout trên Nhánh A (Nuôi Raspberry Pi 4, STM32 & Siêu âm)

- **Vấn đề của nguồn 2S cũ:** Pin 2S 18650 xả xuống đến $6.0\text{V} - 6.4\text{V}$. Module hạ áp buck XL4015 có điện áp sụt tối thiểu (*Dropout Voltage*) $\approx 1.5\text{V}$ ở dòng tải 2A. Khi điện áp pin tụt xuống dưới 6.5V, ngõ ra không thể duy trì 5.15V, khiến điện áp cấp vào Pi 4 rớt dưới ngưỡng bảo vệ 4.63V, gây cảnh báo Undervoltage và reset ngẫu nhiên.
- **Giải pháp nâng cấp 3S 18650:**
- Sử dụng **3 cell 18650 nối tiếp (3S)**: Điện áp đầy 12.6V, điện áp danh định 11.1V, điện áp cắt an toàn 9.0V ($3.0\text{V}/\text{cell}$).
- Tại điểm cắt bảo vệ (9.6V), độ chênh áp vào - ra là: $\Delta V = 9.6\text{V} - 5.15\text{V} = 4.45\text{V} \gg 1.5\text{V}$ (dropout). Mạch XL4015 luôn hoạt động trong vùng điều chỉnh ổn áp lý tưởng, ngõ ra luôn giữ phẳng $5.15\text{V} \pm 0.05\text{V}$.
- Bổ sung **tụ hóa $2200\mu\text{F} / 16\text{V}$ mắc song song tụ gốm 100nF** tại đầu ra của XL4015 để hấp thụ các xung nhọn transient khi CPU Pi 4 đột ngột boost xung nhịp xử lý ảnh OpenCV.
- **Bảo vệ ngõ vào Type-C của Pi 4:**
- Đầu dây cấp nguồn 5.15V vào cổng Type-C của Pi 4 được trang bị thêm **cầu chì tự phục hồi (Polyfuse 3A)** chống quá dòng và **Diode TVS 5V (SMAJ5.0A)** chống xung quá áp.
- Biến trở tinh chỉnh điện áp trên module XL4015 sau khi cân chỉnh chuẩn 5.15V bằng đồng hồ vạn năng được **khóa cố định bằng keo nến hoặc sơn khóa ren** để triệt tiêu nguy cơ rung lắc cơ học làm trôi áp.

B. Nguồn cấp cho Vi điều khiển STM32 Blue Pill

- **Lựa chọn phương án cấp nguồn sạch:** Nguồn nuôi STM32 được trích trực tiếp từ **ngõ ra 5.15V của mạch hạ áp XL4015 (Nhánh A)** đưa vào chân `5V` của bo mạch Blue Pill.
- Dòng tiêu thụ của STM32 Blue Pill rất nhỏ: chỉ $\approx 40 - 50\text{mA}$ khi chạy full xung 72MHz.
- IC ổn áp LDO onboard (RT9193 hoặc AMS1117-3.3) trên bo Blue Pill hạ áp từ 5.15V xuống 3.3V sạch nuôi chip vi điều khiển ARM Cortex-M3 và cấp ra chân 3.3V nuôi 2 module cảm biến đĩa quang LM393.
- *Lý do loại trừ nguồn 5V từ Driver L298N:* Tuyệt đối không lấy nguồn từ chân 5V của L298N (IC 78M05 onboard) vì IC này tỏa nhiệt lớn khi cấp nguồn 7.4V và dễ dẫn nhiễu xung chổi than từ motor sang làm treo vi điều khiển.

C. Mạch Giám sát Pin 3 Cấp Độ & Cơ chế Tắt Mềm (Soft-Shutdown)

- **Mạch cầu phân áp:** Đấu nối từ cực dương khay pin 3S về chân **PA6 (ADC1_Channel_6)** của STM32 qua 2 điện trở chính xác: $R_1 = 10\text{k}\Omega$ (kéo lên Pin (+)) và $R_2 = 3.3\text{k}\Omega$ (kéo xuống Mass).

- Khi pin đầy (12.6V): $V_{ADC} \approx 3.126\text{V} < 3.3\text{V}$ (an toàn tuyệt đối cho chân ADC STM32).
- Khi pin ở mức cảnh báo sớm (10.2V): $V_{ADC} \approx 2.531\text{V}$.
- Khi pin ở mức cưỡng bức tắt máy (9.6V): $V_{ADC} \approx 2.382\text{V}$.

- **Quy trình Cảnh báo 2 Cấp độ & Xử lý Sau Shutdown:**
- **Cấp 1 - Cảnh báo sớm ($V_{bat} \le 10.2\text{V}$, tương đương $3.4\text{V}/\text{cell}$, dung lượng còn $\approx 15 - 20\%$):** STM32 gửi cờ cảnh báo `ALERT_BATTERY_LOW` lên Web dashboard, bật còi bíp ngắt quãng. Xe hoàn thành nốt lộ trình đến trạm hiện tại rồi khóa nhận nhiệm vụ mới để bảo vệ pin.
- **Cấp 2 - Cưỡng bức Shutdown ($V_{bat} \le 9.6\text{V}$, tương đương $3.2\text{V}/\text{cell}$, dung lượng còn $\approx 5 - 8\%$):** STM32 lập tức phanh dừng xe, gửi gói tin khẩn cấp `0x04 (CMD_FORCED_SHUTDOWN)` qua UART lên Pi 4. Pi 4 ghi log trạng thái và gọi lệnh `sudo shutdown -h now`. Ở mức 9.6V, mạch XL4015 vẫn có biên độ điện áp $\Delta V = 4.45\text{V} \gg 1.5\text{V}$ dropout, đảm bảo Pi 4 có dư dả $20 - 30\text{ giây}$ để hoàn tất lưu trữ dữ liệu an toàn mà không bao giờ bị brownout.
- **Xử lý sau khi Pi tắt (Bảo vệ xả sâu):** Khi Pi 4 hoàn tất shutdown (trạng thái halt), bo mạch Pi vẫn tiêu thụ $\approx 50 - 100\text{mA}$. STM32 duy trì bộ đếm thời gian trễ **40 giây** sau khi phát lệnh shutdown (đảm bảo đèn LED xanh ACT trên Pi 4 đã ngừng nhấp nháy), sau đó kích hoạt còi bíp liên tục cảnh báo kỹ sư đến gạt công tắc cơ (Công tắc A) để ngắt hoàn toàn nguồn điện, chống xả kiệt pin.

---

2. Phân tích Dòng Tải Động Cơ & Driver Công Suất

A. Đánh giá Khả năng Chịu Tải của L298N & Giới hạn Thực tế

- **Đặc tính dòng của 4 động cơ TT:**
- Dòng không tải: $\approx 150 - 200\text{mA}$ mỗi motor.
- Dòng hoạt động bình thường trên sàn lab có tải (khung xe + pin + Pi 4 $\approx 1.2\text{kg}$): $\approx 350 - 450\text{mA}$ mỗi motor.
- Hai motor mỗi bên mắc song song vào một kênh cầu H của L298N: Dòng làm việc danh định của mỗi kênh là $0.7\text{A} - 0.9\text{A}$.
- Dòng kẹt (Stall Current): Khi bánh xe bị kẹt hoàn toàn, dòng stall có thể đạt $1.2\text{A} - 1.5\text{A}$ mỗi motor, dẫn tới dòng đỉnh tức thời của 1 kênh lên tới $2.4\text{A} - 3.0\text{A}$.
- **Biện pháp kỹ thuật kiểm soát trên L298N:**
- *Giải nhiệt phần cứng:* Trang bị cánh tản nhiệt nhôm lớn đi kèm module L298N, cố định tại vị trí thông thoáng trên khung xe để tản nhiệt cho transistor BJT Darlington.
- *Giải thuật Khởi động mềm (Soft-start Ramp):* Phần mềm STM32 không cấp xung 100% PWM đột ngột mà tăng dần độ rộng xung với tốc độ dốc không quá $20\% / 100\text{ms}$. Điều này triệt tiêu dòng inrush lúc bắt đầu chuyển động.
- *Bảo vệ kẹt bánh qua Encoder:* Nếu STM32 phát lệnh PWM lớn hơn PWM_{min} mà sau 500ms không nhận được bất kỳ xung encoder nào (xe bị kẹt chân bàn/chướng ngại vật), STM32 tự động ngắt PWM về 0 để chống quá nhiệt cầu H.
- *Lưu ý về sụt áp Darlington:* Do transistor BJT Darlington trên L298N có sụt áp nội $1.6\text{V} - 2.0\text{V}$, động cơ TT từ pin 7.4V thực tế chỉ nhận được $\approx 5.4\text{V} - 5.6\text{V}$. Các giá trị duty cycle (PWM_{min}, PWM_0) trong tài liệu là **giá trị khởi điểm giả định** và bắt buộc phải được chuẩn hóa lại thông qua đo đạc thực nghiệm trên sàn lab.

- **Đề xuất nâng cấp tối ưu (Khuyến nghị phần cứng):**
- Thay thế L298N bằng **module TB6612FNG** (hoặc 2 module TB6612FNG riêng biệt). Cầu H MOSFET của TB6612FNG có nội trở cực thấp $R_{DS(on)} \approx 0.5\Omega$, sụt áp chỉ $\approx 0.2\text{V} - 0.3\text{V}$, hiệu suất năng lượng $>90\%$, mát rượi và chịu dòng đỉnh 3.2A an toàn hơn hẳn BJT L298N.

---

3. Giao tiếp UART Ổn Định giữa Pi 4 và STM32

A. Lựa chọn Phần cứng: Mạch USB-TTL CP2102 (Phương án Tối ưu Số 1)

- **Vấn đề của cổng GPIO 14/15 mặc định trên Pi 4:** Trên hệ điều hành Raspberry Pi OS mặc định, cổng GPIO14/15 được gán cho `mini-UART` (`/dev/ttyS0`), có xung nhịp phụ thuộc trực tiếp vào tần số CPU của Pi 4. Khi CPU thay đổi tần số (Dynamic Frequency Scaling), baudrate bị sai lệch nhẹ gây lỗi mất byte ngẫu nhiên ở tốc độ 115200 baud. Trong khi đó, UART phần cứng PL011 chất lượng cao (`/dev/ttyAMA0`) lại mặc định được gán cho chip Bluetooth.
- **Giải pháp chuẩn xác:** Sử dụng mạch nạp **USB-TTL (chip CP2102 hoặc CH340)** cắm trực tiếp vào cổng USB 2.0 của Raspberry Pi 4.
- Cổng giao tiếp trong Linux nhận diện là `/dev/ttyUSB0`.
- Module USB-TTL tự cấp nguồn 5V từ cổng USB của Pi 4, không cần kéo dây nguồn phụ từ pin.
- Module sở hữu thạch anh dao động độc lập, tạo xung nhịp baudrate 115200 cực kỳ chuẩn xác, jitter gần như bằng 0, hoàn toàn không bị ảnh hưởng bởi tải CPU của Pi 4 và không yêu cầu chỉnh sửa phức tạp cấu hình Linux.
- Kết nối 3 dây sang STM32: Chân TX của CP2102 nối vào RX1 (PA10), chân RX nối vào TX1 (PA9), chân GND nối chung mass.
- **Phương án dự phòng (Nếu dùng Header GPIO 14/15):** Cần cấu hình vô hiệu hóa Bluetooth để gán PL011 cho GPIO14/15: `ini # Thêm vào cuối file /boot/firmware/config.txt (hoặc /boot/config.txt) enable_uart=1 dtoverlay=disable-bt` Và chạy lệnh tắt service Bluetooth: `sudo systemctl disable hciuart`, sau đó sử dụng port `/dev/ttyAMA0`.

---

4. Cấp nguồn & Lọc nhiễu cho 3 Cảm biến Siêu âm HC-SR04

- **Đặc tính nguồn của HC-SR04:** Cảm biến đòi hỏi điện áp nuôi chuẩn $5.0\text{V} \pm 0.5\text{V}$ (không hoạt động ở mức 3.3V).
- **Thiết kế đường nguồn sạch:**
- Nguồn nuôi VCC của 3 cảm biến HC-SR04 được trích từ **ngõ ra 5.15V sạch của mạch hạ áp XL4015 (Nhánh A)**, hoàn toàn không câu từ nguồn pin động cơ 7.4V để tránh sụt áp và xung nhiễu chổi than.
- Tổng dòng tiêu thụ của 3 cảm biến: $3 \times 15\text{mA} = 45\text{mA}$ (chiếm chưa đầy 1% công suất danh định 5A của XL4015).
- **Lọc nhiễu cục bộ:** Tại chân VCC/GND của từng module HC-SR04, lắp một cặp **tụ lọc gồm tụ hóa $100\mu\text{F} / 16\text{V}$ song song tụ gốm 100nF** để san phẳng các xung sụt áp tức thời trong thời điểm phát xung siêu âm 40kHz.

- **Bảo vệ chân GPIO STM32:**
- Chân phát `Trig` nối với GPIO STM32 cấu hình Output Push-Pull mức 3.3V (đạt ngưỡng kích hoạt $V_{IH} \ge 2.0\text{V}$ của HC-SR04).
- Chân thu `Echo` (ngõ ra 5V) nối trực tiếp vào các chân **PB13, PB15, PB9** của STM32F103C8T6. Đây là các chân có cấu trúc **5V-Tolerant (FT)** theo đúng datasheet của STMicroelectronics, chịu được điện áp 5V an toàn mà không cần mạch đệm hay cầu phân áp điện trở.

---

5. Làm rõ Tài nguyên Phần cứng & Cấu hình Điện trở Kéo

- **Chân PB6/PB7 (I2C1 vs EXTI):** Trên STM32F103C8T6, chân **PB6** và **PB7** mặc định là chân ngoại vi I2C1. Trong đề tài này, màn hình OLED hiển thị trạng thái được cắm vào cổng **I2C của Raspberry Pi 4** (GPIO 2 và GPIO 3), do đó ngoại vi I2C1 trên STM32 được giải phóng hoàn toàn và dùng làm chân ngắt ngoài **EXTI6** và **EXTI7** đếm xung encoder.
- **Điện trở kéo lên (Pull-up) cho Encoder:** Điện trở kéo lên nội của STM32 có giá trị khá lớn ($\approx 40\text{k}\Omega$), không đủ mạnh để tạo sườn xung vuông sắc nét cho ngõ ra open-collector nếu dây nối dài. Module đĩa quang LM393 thương mại đã tích hợp sẵn mạch so sánh LM393 và điện trở kéo lên Pull-up $10\text{k}\Omega$ trên bo. Trong trường hợp dây bus kéo dài $> 15\text{cm}$, nhóm hàn bổ sung cặp **điện trở kéo ngoài $4.7\text{k}\Omega - 10\text{k}\Omega$ lên đường 3.3V** sát chân STM32 để chống nhiễu sườn xung.
- **Xác nhận Bus I2C trên Pi 4:** Màn hình OLED 0.91" SSD1306 sử dụng bus phần cứng `i2c-1` trên GPIO 2 (SDA) và GPIO 3 (SCL) của Pi 4 tại địa chỉ `0x3C`. Hệ thống không gắn thêm bất kỳ bo mở rộng (HAT) nào khác nên hoàn toàn không xảy ra xung đột bus I2C.
- **Nút dừng khẩn cấp (E-Stop):** Nút nhấn nhả cơ khí to được đấu nối tiếp trên đường cấp nguồn 7.4V (VM) của Driver L298N. Khi nhấn E-Stop, chỉ duy nhất nguồn động lực cấp cho động cơ bị ngắt tức thời, **toàn bộ khối máy tính nhúng Pi 4 và STM32 vẫn giữ nguyên nguồn điện hoạt động bình thường** để tiếp tục ghi log và truyền dữ liệu telemetry lên Web. Khi nhả E-Stop, hệ thống không tự động chạy tiếp mà phần mềm yêu cầu người dùng bấm nút xác nhận "Khởi động lại / Reset" trên giao diện Web.

PHẦN 3: GIAO THỨC TRUYỀN THÔNG, THUẬT TOÁN ĐIỀU KHIỂN & FSM

1. Giao thức Truyền thông UART Nhị phân Có Checksum & Watchdog

Để đảm bảo robot không bao giờ bị mất kiểm soát khi máy tính nhúng Pi 4 bị quá tải hoặc treo phần mềm, hệ thống áp dụng giao thức truyền thông nhị phân định dạng gói cố định (*Binary Packet Protocol*) kết hợp cơ chế **Watchdog Timer an toàn**.

A. Cấu trúc Khung Gói tin (Frame Format)

Mỗi gói tin truyền qua UART có định dạng chuẩn:

```
┌──────┬──────┬─────┬──────────┬──────────────────────┬──────────┐
│ 0xAA │ 0x55 │ LEN │ MSG_TYPE │       PAYLOAD        │ CHECKSUM │
└──────┴──────┴─────┴──────────┴──────────────────────┴──────────┘
  1B     1B     1B      1B        (LEN - 1) Bytes         1B
```

- **Header (2 Bytes):** `0xAA 0x55` — Đồng bộ hóa điểm bắt đầu khung truyền, lọc bỏ byte rác do nhiễu đường truyền.
- **LEN (1 Byte):** Độ dài của phần dữ liệu tính từ `MSG_TYPE` đến hết `PAYLOAD`.
- **MSG_TYPE (1 Byte):** Mã định danh lệnh:
- `0x01` (`CMD_VELOCITY`): Gửi từ Pi 4 $\rightarrow$ STM32. Payload gồm 4 bytes: vận tốc bánh trái v_L (int16_t, phạm vi $-100 \dots +100\%$) và vận tốc bánh phải v_R (int16_t).
- `0x02` (`CMD_HEARTBEAT`): Gửi từ Pi 4 $\rightarrow$ STM32 khi xe đang đứng yên hoặc duy trì trạng thái (Payload 0 byte).
- `0x03` (`CMD_EMERGENCY_STOP`): Lệnh dừng khẩn cấp từ phần mềm Web (Payload 0 byte).
- `0x81` (`TELEMETRY_DATA`): Gửi từ STM32 $\rightarrow$ Pi 4 (chu kỳ 10Hz). Payload gồm 8 bytes: 3 khoảng cách siêu âm Trái - Giữa - Phải (3x uint8_t cm), điện áp pin (uint16_t mV), xung encoder Trái và Phải (2x uint16_t).
- `0x04` (`ALERT_LOW_BATTERY`): Gửi từ STM32 $\rightarrow$ Pi 4 khi điện áp pin rơi dưới 9.6V để kích hoạt soft-shutdown.
- **CHECKSUM (1 Byte):** Thuật toán XOR Checksum tính trên toàn bộ các byte từ `LEN` đến hết byte cuối cùng của `PAYLOAD`:

B. Cơ chế Watchdog Timer An toàn 300ms trên STM32

- Máy tính nhúng Pi 4 duy trì chu kỳ gửi lệnh điều khiển hoặc Heartbeat định kỳ mỗi **$50\text{ms} - 100\text{ms}$** ($10\text{Hz} - 20\text{Hz}$).
- Vi điều khiển STM32 khởi tạo một bộ đếm thời gian phần mềm `watchdog_ms` (được gia tăng 1ms trong ngắt `SysTick_Handler`).
- Mỗi khi bộ giải mã UART nhận đúng gói tin hợp lệ (đúng Header và khớp Checksum), `watchdog_ms` được reset về 0.
- **Xử lý sự cố mất kết nối hoặc Pi 4 bị treo:** Nếu `watchdog_ms > 300ms` (tương đương mất liên tiếp 3 - 5 khung truyền):
- STM32 lập tức kích hoạt hàm `SAFETY_MOTOR_CUT()`: ép toàn bộ xung $PWM_L = PWM_R = 0$, khóa các chân cầu H để hãm động năng xe.
- STM32 tự động kích hoạt còi báo động tiếng bíp ngắt quãng liên tục để kỹ sư trong phòng phát hiện vị trí robot đang dừng giữa hành lang (vì lúc này Web dashboard không thể cập nhật do Pi 4 đã bị treo).

---

2. Động học Xe, Giới hạn Encoder 1 Kênh & Quy tắc Odometry

A. Thông số Cơ khí & Tính toán Vận tốc

- **Động cơ TT:** Hộp số giảm tốc $1:48$. Điện áp cấp danh định sau sụt áp L298N từ pin 7.4V là $\approx 5.4\text{V} - 5.6\text{V}$.
- Tốc độ không tải ở 5.5V: $\approx 160 - 180\text{ RPM}$.
- Bánh xe cao su đường kính $D = 65\text{mm} = 0.065\text{m}$. Chu vi bánh xe: $C = \pi \times D \approx 0.204\text{m} = 20.4\text{cm}$.
- Vận tốc tối đa lý thuyết khi có tải: $v_{max} \approx 0.30 - 0.35\text{ m/s}$.
- **Điểm đặt vận tốc vận hành:**
- **Vận tốc hành trình (Cruising Speed):** Đặt mục tiêu ban đầu $v_0 \approx 0.20\text{ m/s}$ (tương ứng tốc độ quay bánh $\approx 60\text{ RPM}$, duty cycle $PWM_0 \approx 50\%$).
- **Vận tốc tiếp cận trạm (Docking Speed):** Đặt mục tiêu ban đầu $v_{dock} \approx 0.10 - 0.12\text{ m/s}$ ($PWM \approx 40\% - 45\%$). Không đặt PWM quá thấp dưới 35% vì motor TT dễ bị kẹt ma sát tĩnh hoặc quay giật cục. Mọi con số PWM đều là giá trị khởi điểm và sẽ được hiệu chuẩn bằng thực nghiệm trên mặt sàn cụ thể.

B. Phân tích Giới hạn của Encoder 1 Kênh LM393

- **Đặc tính đĩa quang:** Đĩa quang 20 nan tạo ra 20 xung trên một vòng quay của bánh xe ($C \approx 20.4\text{cm}$).
- Độ phân giải không gian: $1\text{ xung} \approx 1.02\text{cm}$.
- Tại vận tốc $v_0 = 0.20\text{ m/s}$, tần số xung thu được là $\approx 20\text{ xung/giây}$. Tại vận tốc chậm $v_{dock} \approx 0.10\text{ m/s}$, tần số xung chỉ $\approx 10\text{ xung/giây}$.
- **Thừa nhận giới hạn trung thực:** Module cảm biến quang chữ U LM393 chỉ có 1 kênh xung D0 (không có tín hiệu kênh B lệch pha 90° như Quadrature Encoder), do đó **hoàn toàn không xác định được chiều quay tiến hay lùi**. Khi xe bị trượt bánh hoặc bị ngoại lực đẩy lùi, bộ đếm vẫn tăng xung dương.
- **Ứng dụng chuẩn xác trong đề tài:**
- **Cân bằng vận tốc tiến (Forward Velocity Balancing):** Encoder chỉ được sử dụng trong thuật toán PI vi sai để bù trừ sự sai lệch cơ khí giữa động cơ bên trái và bên phải khi xe chạy thẳng, giúp robot duy trì quỹ đạo ổn định.
- **Giới hạn Dead-Reckoning:** Khi xe bị mất mốc trong trạng thái tiếp cận trạm, thời gian chạy mù theo đếm xung encoder được giới hạn nghiêm ngặt **$t_{dead} \le 0.5\text{ giây}$** (tương đương quãng đường di chuyển $< 10\text{cm}$). Tuyệt đối không dùng encoder 1 kênh cho các bài toán định vị tọa độ toàn cục lâu dài.

---

3. Hình học Quang học Camera Logitech C270 & Chuẩn hóa Mốc ArUco

A. Cố định Kích thước Vật lý & Vị trí Mốc ArUco

- Kích thước mốc ArUco được chuẩn hóa cố định là:

í ướ ố

- **Quy cách bố trí hình học:**
- Mốc thuộc từ điển `cv2.aruco.DICT_4X4_50`, in trên giấy decal mờ (giảm phản xạ lóa đèn), dán trên tấm mica cứng gắn tại trạm, viền trắng bao quanh $\ge 2\text{cm}$.
- **Độ cao tâm mốc:** Dán cố định ở độ cao **$25 - 30\text{cm}$** so với mặt sàn.
- **Góc đặt camera trên xe:** Camera Logitech C270 bắt cố định ở độ cao $\approx 20\text{cm}$ trên tầng 2 thân xe và nghiêng nhẹ một góc $5^\circ - 8^\circ$ hướng lên trên để trục quang học của camera hướng thẳng vào tâm mốc ở mọi cự ly từ 2.0m đến 0.5m.

B. Phân tích Mô hình Pinhole Camera

Webcam Logitech C270 hoạt động ở độ phân giải 640×480 pixel. Góc nhìn ngang thực tế $HFOV \approx 55^\circ$. * Tiêu cự pixel tương đương theo phương ngang:

(Thực tế hiệu chuẩn dao động trong khoảng $560 - 615\text{ pixels}$ tùy khoảng cách lấy nét). * Theo phương trình hình chiếu tâm:

(Trong đó: W_{px} là bề rộng mốc tính bằng pixel trên ảnh; $W_{real} = 18\text{cm}$; Z là khoảng cách thực tế từ camera tới mốc).

C. Bảng Tương quan Kích thước Pixel & Khoảng cách Thực tế

Với giá trị thực nghiệm trung bình $f_x \approx 560\text{ pixels}$:

Khoảng cách thực tế Z	Bề rộng mốc trên ảnh (W_{px})	Ý nghĩa trạng thái hệ thống
200cm (2.0m)	$\approx 50\text{ pixels}$	Bắt đầu phát hiện mốc trong trạng thái `SEARCH`
100cm (1.0m)	$\approx 101\text{ pixels}$	Trạng thái bám mốc `NAVIGATE` ổn định
55cm	$\approx 183\text{ pixels}$	**Ngưỡng kích hoạt chuyển sang** `APPROACH_DOCKING` ($W_{marker} \ge 180\text{px}$)
30cm	$\approx 336\text{ pixels}$	Xe bò chậm, mốc chiếm quá nửa khung hình
$\le 15\text{cm}$	$> 500\text{ pixels}$	Mốc bắt đầu tràn biên ảnh; cảm biến siêu âm tiếp quản hoàn toàn

D. Quy trình Hiệu chuẩn Camera (Intrinsic Calibration)

- Nhóm thực hiện hiệu chuẩn nội tại camera bằng bàn cờ chuẩn Checkerboard 8×6 (kích thước mỗi ô vuông 25mm).
- Thu nhận 25 khung hình ở các góc nghiêng và khoảng cách khác nhau, sử dụng thuật toán `cv2.calibrateCamera()` để trích xuất ma trận camera K và vector méo quang học D.
- Ảnh được khử méo (*undistort*) trước khi nhận diện ArUco giúp nâng cao độ chính xác định vị tâm mốc lên dưới $\pm 3\text{ pixels}$.

---

4. Thuật toán Lái Visual Servoing bằng Bộ Điều Khiển PD Chuẩn Hóa

A. Cơ sở Lý thuyết Điều khiển

Hệ thống điều khiển góc lái bám tâm mốc là hệ tích phân bậc 1: Sai số góc lệch tâm $\rightarrow$ Vận tốc quay vi sai của xe $\rightarrow$ Tích phân thành góc hướng của thân xe. Hiện tượng dao động trong thực nghiệm phát sinh từ độ trễ vòng lặp điều khiển ($T_{delay} \approx 60 - 80\text{ms}$ gồm: trễ camera UVC, trễ OpenCV, trễ truyền thông UART và quán tính cơ khí). Việc áp dụng bộ điều khiển PD chuẩn hóa bổ sung thành phần vi phân K_d tạo hiệu ứng giảm chấn (*damping*), hãm tốc độ quay khi tâm mốc tiến gần về trục giữa của xe.

B. Công thức Toán học Chuẩn hóa Không Thứ nguyên

Độ phân giải khung hình 640×480 pixel. Tọa độ tâm quang học $X_{center} = 320$.
Tại mỗi chu kỳ xử lý ảnh thứ k với timestamp thực tế t_k: 1. **Sai số chuẩn hóa không thứ nguyên (đưa về đoạn $[-1.0, 1.0]$):**

---

2. **Chu kỳ lấy mẫu thực tế đo bằng `time.perf_counter()`:**

3. **Đạo hàm sai số qua Bộ lọc thông thấp số (First-order Low-pass Filter):**

ọ

4. **Tín hiệu điều khiển vi sai có bão hòa (Saturation):**

(Bộ thông số ban đầu hiệu chuẩn: $K_p \approx 0.45$, $K_d \approx 0.08$). 5. **Phân bổ duty cycle xung PWM (%) tới 2 vế bánh xe:**

(Trong đó: $PWM_0 = 50\%$; $PWM_{scale} = 30\%$; $PWM_{min} \approx 35 - 40\%$; $PWM_{max} = 80\%$).

---

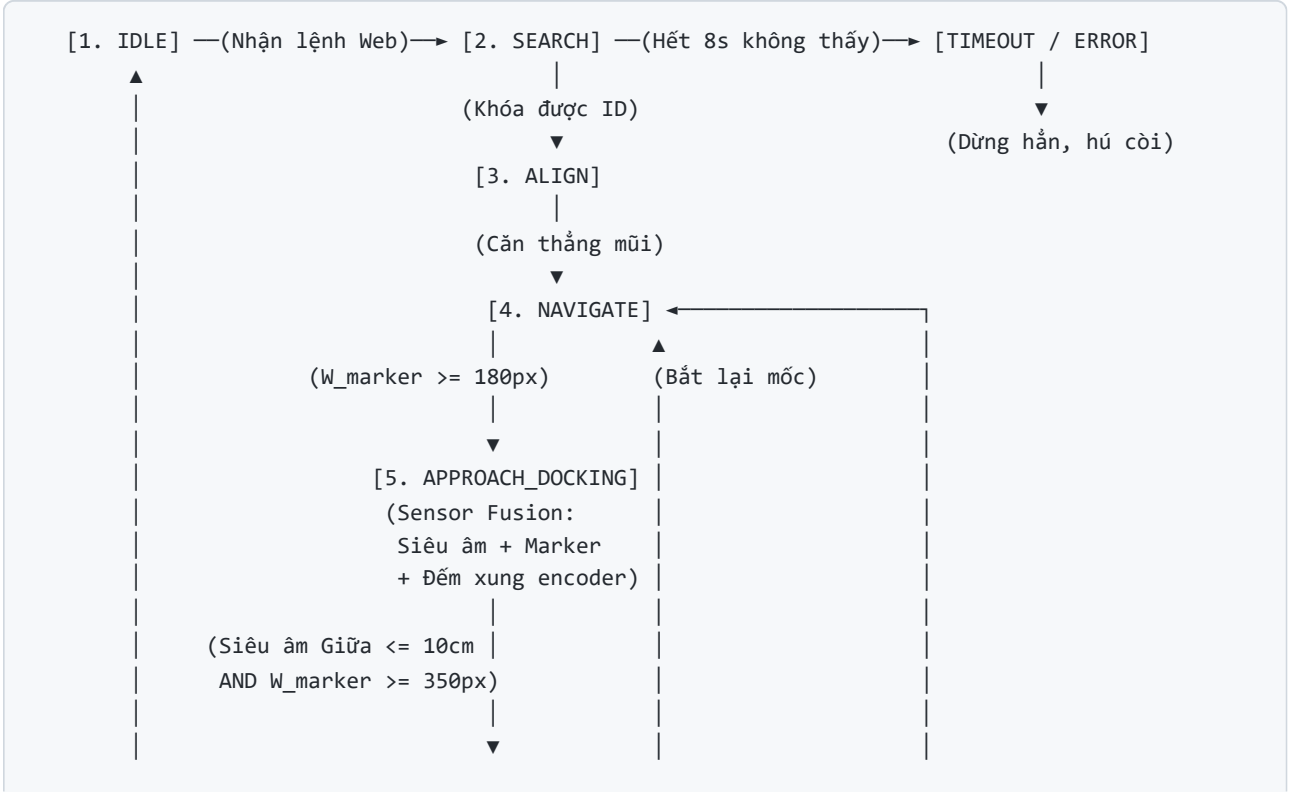
5. Giải thuật Chống Nhiễu Chéo 3 Cảm biến Siêu âm (Time-Division Multiplexing - TDM)

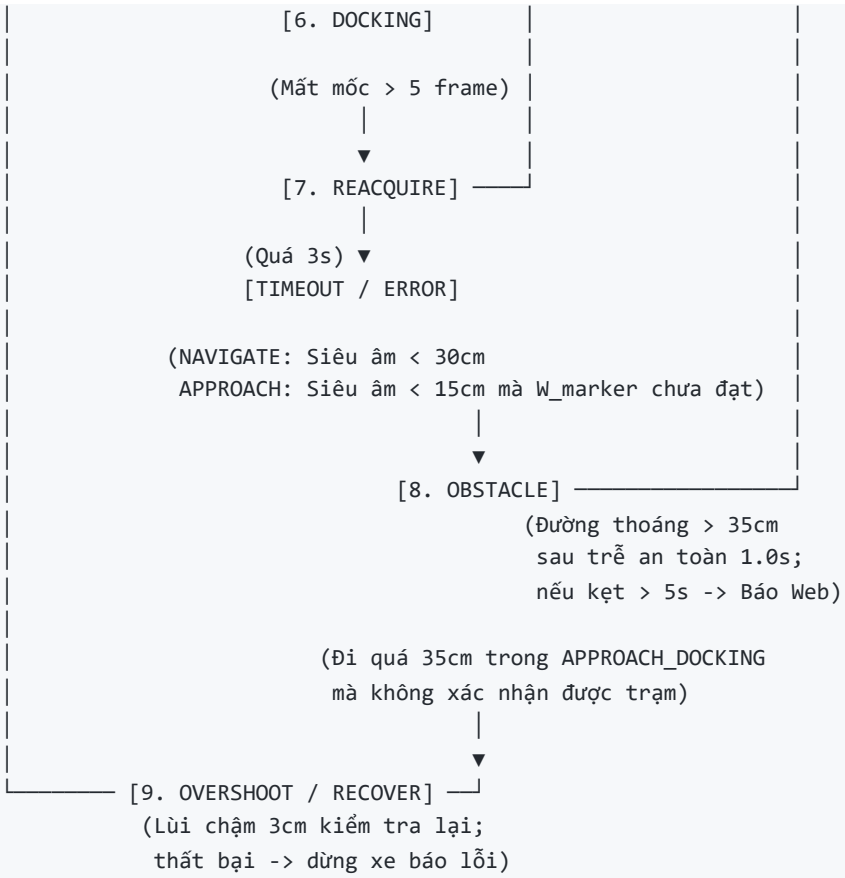
- **Nguyên lý vật lý:** Sóng siêu âm phát ra có góc mở $\approx 15^\circ - 20^\circ$. Nếu kích phát đồng thời cả 3 cảm biến Trái, Giữa, Phải, sóng phản xạ từ cảm biến này sẽ dội chéo vào đầu thu của cảm biến kia gây ra hiện tượng đo sai khoảng cách và kích hoạt các phản xạ phanh ảo ("phanh ma").
- **Phân chia thời gian (TDM):** Vận tốc âm thanh trong không khí $v \approx 343\text{ m/s}$. Với cự ly đo giới hạn tối đa của HC-SR04 là 4.0m, thời gian khứ hồi tối đa là:

- **Chu kỳ quét tuần tự:**
- **0ms:** Kích xung cảm biến Trái $\rightarrow$ chờ 25ms để sóng dội tiêu tán hoàn toàn.
- **25ms:** Kích xung cảm biến Giữa $\rightarrow$ chờ 25ms.
- **50ms:** Kích xung cảm biến Phải $\rightarrow$ chờ 25ms.
- **Tổng chu kỳ quét trọn vẹn 3 cảm biến:** 75ms ($\approx 13.3\text{ Hz}$). Ở tốc độ hành trình 0.20 m/s, xe chỉ di chuyển được 1.5cm trong mỗi chu kỳ quét, đảm bảo phát hiện vật cản kịp thời mà loại bỏ 100% xung đột sóng âm.

---

6. Chu trình Máy Trạng Thái Hữu Hạn (FSM) Hoàn Chỉnh





1. `IDLE` **(Chờ lệnh):** Xe đứng yên tại trạm xuất phát, màn hình OLED hiển thị IP mạng Wi-Fi và pin. Giao diện Web chờ lệnh vận chuyển.
2. `SEARCH` **(Quét tìm mốc bằng thân xe):** Xe thực hiện xoay vi sai từng bước (*Step-Turn-and-Check*): xoay nhích $15^\circ - 20^\circ \rightarrow$ dừng tĩnh 100ms để chụp khung hình không rung:
3. Nếu phát hiện đúng mã ID yêu cầu: chuyển sang `ALIGN`.
4. Nếu quay quá 1 vòng tròn ($> 8\text{ giây}$): dừng hẳn, chuyển sang `TIMEOUT / ERROR` và báo lên Web.
5. `ALIGN` **(Căn chỉnh trục giữa):** Robot nhích nhẹ thân xe tại chỗ cho tới khi tâm mốc nằm sát tâm camera ($|e_{norm}| < 0.1$) $\rightarrow$ Chuyển sang `NAVIGATE`.
6. `NAVIGATE` **(Hành trình bám mốc Visual Servoing):**
7. Xe tiến với vận tốc hành trình $v_0 = 0.20\text{ m/s}$ kết hợp thuật toán lái PD chuẩn hóa.
8. Cụm siêu âm giám sát liên tục: nếu bất kỳ cảm biến nào đo được $< 30\text{cm}$ $\rightarrow$ ngắt PWM tức thì, chuyển sang `OBSTACLE`.
9. **Điều kiện chuyển trạng thái tiếp cận:** Khi kích thước mốc ArUco đạt $W_{marker} \ge 180\text{px}$ (tương ứng cự ly $\approx 55\text{cm}$) $\rightarrow$ Chuyển sang `APPROACH_DOCKING`.
10. **Xử lý mất dấu tạm thời (Lost Marker Counter):** Mất $1 - 4$ frame: xe giữ nguyên hướng lái ở tốc độ chậm; Nếu mất liên tiếp ≥ 5 frame ($> 250\text{ms}$): xe dừng hẳn, chuyển sang `REACQUIRE` (nhích nhẹ thân xe $\pm 15^\circ$ tìm lại). Nếu quá 3.0s không thấy mốc $\rightarrow$ Dừng hẳn và cảnh báo lỗi.
11. `APPROACH_DOCKING` **(Tiếp cận trạm cận đích - Hợp nhất Cảm biến):**

12. *Giải quyết triệt để xung đột giữa Bảng trạm và Vật cản:*
    1. Xe hạ tốc độ về mức bò an toàn $v_{dock} \approx 0.10 - 0.12\text{ m/s}$ ($PWM \approx 40\% - 45\%$).
    2. Trong trạng thái này, chỉ sử dụng cảm biến siêu âm Giữa để đo khoảng cách cập trạm (2 cảm biến bên hông vẫn giám sát vật cản chéo $< 20\text{cm}$).
    3. **Phân biệt Trạm và Vật cản đột xuất:** Nếu cảm biến siêu âm giữa đo được $< 15\text{cm}$ nhưng kích thước mốc trên ảnh $W_{marker} < 350\text{px}$ (mốc chưa đủ lớn) $\rightarrow$ Đây là vật thể lạ chen vào giữa xe và trạm (chân người, thùng hàng) $\rightarrow$ Chuyển ngay sang `OBSTACLE` để đảm bảo an toàn!
    4. Nếu khoảng cách siêu âm giữa không giảm đều trong 3 chu kỳ liên tiếp dù xe đang phát PWM $\rightarrow$ Xe bị vướng cản $\rightarrow$ Chuyển sang `OBSTACLE`.
    5. Đếm xung encoder giới hạn quãng đường bò tối đa: $\le 35\text{cm}$.
13. **Điều kiện cập trạm thành công:** Khi cảm biến siêu âm giữa đo được **$\le 10\text{cm}$** ĐỒNG THỜI $W_{marker} \ge 350\text{px}$ ĐỒNG THỜI quãng đường encoder hợp lệ $\rightarrow$ Kích hoạt phanh dừng tuyệt đối, chuyển sang `DOCKING`.
14. `DOCKING` **(Cập trạm thành công):** Ngắt toàn bộ xung PWM, phát chuỗi còi bíp dài thông báo cho kỹ sư lấy khay linh kiện, cập nhật trạng thái lên Web dashboard và đưa hệ thống trở lại `IDLE`.
15. `OBSTACLE` **(Phản xạ né tránh vật cản):** STM32 ngắt PWM tức thời ($< 20\text{ms}$), phát còi bíp cảnh báo. Khi vật cản rời đi (khoảng cách $> 35\text{cm}$), bộ lọc giữ trạng thái chờ thêm **1.0 giây** để đảm bảo đường đi hoàn toàn thông thoáng trước khi tiếp tục bám mốc. Nếu vật cản chặn đường quá **5.0 giây** $\rightarrow$ Chuyển sang trạng thái cảnh báo kẹt đường và gửi thông báo lên Web.
16. `OVERSHOOT / RECOVER` **(Xử lý vượt trạm hoặc mất tín hiệu cận đích):**
17. Nếu xe đã di chuyển hết quãng đường quy định (35cm theo xung encoder) trong trạng thái `APPROACH_DOCKING` mà cảm biến siêu âm vẫn không nhận được cự ly $\le 10\text{cm}$ (do góc tới của tia siêu âm bị xiên), hoặc mốc ArUco đột ngột biến mất:
18. Robot lập tức phanh dừng, chuyển sang lùi chậm 3cm trong 200ms để tái quan sát. Nếu vẫn không thể xác định khoảng cách an toàn, xe dừng hẳn và báo lỗi lên Web, tuyệt đối không đâm va vào bàn trạm.

PHẦN 4: BẢNG DANH MỤC LINH KIỆN MUA SẮM (BOM) & DỰ TOÁN CHÍNH XÁC

(Đã tận dụng tối đa các thiết bị có sẵn trong phòng thí nghiệm: Raspberry Pi 4 8GB, Mạch nạp USB-TTL CP2102 và Webcam Logitech C270 HD).

STT	Tên linh kiện	Model / Thông số kỹ thuật	SL	Đơn giá thực tế	Thành tiền	Ghi chú kỹ thuật
1	**Bộ xử lý trung**	Raspberry Pi 4 Model B (Bản 8GB RAM)	1	*0đ (Đã có)*	**0đ**	Chạy Linux, OpenCV ArUco,

STT	Tên linh kiện	Model / Thông số kỹ thuật	SL	Đơn giá thực tế	Thành tiền	Ghi chú kỹ thuật
	tâm (Master)					Web Flask
2	**Camera nhận diện mốc**	Webcam USB Logitech C270 HD (720p)	1	*0đ (Đã có)*	**0đ**	Bắt cố định vào khung xe, cắm cổng USB Pi 4
3	**Mạch giao tiếp UART liên tầng**	Module USB-TTL UART (Chip CP2102)	1	*0đ (Đã có)*	**0đ**	Cắm USB Pi 4 (`/dev/ttyUSB0`), thạch anh riêng
4	**Vi điều khiển thời gian thực (Slave)**	STM32F103C8T6 Blue Pill (Type-C)	1	48.000đ	48.000đ	Chạy ARM Cortex-M3 72MHz, điều khiển PWM
5	**Cảm biến đĩa quang (Encoder)**	Module đĩa quang chữ U LM393	2	8.000đ	16.000đ	Đọc xung D0 qua ngắt EXTI PB6/PB7
6	**Cảm biến khoảng cách**	Module siêu âm HC-SR04	3	18.000đ	54.000đ	Bố trí Trái – Giữa – Phải (quét TDM 25ms)
7	**Màn hình hiển thị tại chỗ**	OLED 0.91 inch I2C (SSD1306)	1	28.000đ	28.000đ	Cắm cổng I2C của Pi 4 hiển thị IP và Pin
8	**Khung xe robot 4 bánh**	Khung mica 2 tầng chữ nhật (kèm 4 motor TT)	1 bộ	115.000đ	115.000đ	Đi kèm 4 đĩa encoder đen 20 nan
9	**Mạch công suất động cơ**	Module cầu H L298N (có tản nhiệt nhôm lớn)	1	32.000đ	32.000đ	Điều khiển 4 motor TT qua 2 kênh
10	**Nút dừng khẩn cấp cơ khí**	Nút nhấn nhả to (E-Stop Push Button)	1	5.000đ	5.000đ	Đấu nối tiếp đường nguồn pin cấp cho L298N
11	**Còi báo động + Transistor đệm**	Active Buzzer 5V + Transistor 2N2222 + Trở 1k	1 bộ	4.000đ	4.000đ	Đệm dòng bảo vệ chân GPIO PB5 của STM32

STT	Tên linh kiện	Model / Thông số kỹ thuật	SL	Đơn giá thực tế	Thành tiền	Ghi chú kỹ thuật
12	**Khoang chứa hàng**	Hộp nhựa mini có nắp	1	10.000đ	10.000đ	Đặt ở nửa sau thân xe để chở khay linh kiện
13	**Pin sạc Li-ion 18650**	Cell 18650 xả cao $\ge 10\text{A}$ (2200-2600mAh)	5 viên	32.000đ	160.000đ	3 viên cho Nhánh A (Pi), 2 viên cho Nhánh B (Motor)
14	**Khay pin 3 cell (3S)**	Khay 3 pin 18650 nối tiếp có dây ra	1 cái	12.000đ	12.000đ	Cấp nguồn 11.1V cho mạch Buck hạ áp Pi 4
15	**Khay pin 2 cell (2S)**	Khay 2 pin 18650 nối tiếp có dây ra	1 cái	10.000đ	10.000đ	Cấp nguồn 7.4V cho mạch công suất động cơ
16	**Mạch hạ áp Buck DC-DC**	Module XL4015 (5A có tản nhiệt nhôm)	1	28.000đ	28.000đ	Hạ từ 11.1V xuống đúng **5.15V** sạch nuôi Pi 4
17	**Linh kiện lọc nguồn & Đo áp**	Tụ $2200\mu\text{F}/16\text{V}$, 3 tụ $100\mu\text{F}+100\text{nF}$, trở $10\text{k}+3.3\text{k}$, TVS 5V, Polyfuse 3A	1 gói	6.000đ	6.000đ	Lọc nguồn Pi, lọc nguồn 3 siêu âm, phân áp ADC, bảo vệ
18	**Công tắc nguồn**	Công tắc bập bênh nhỏ 2 chân	2 cái	3.000đ	6.000đ	Bật/tắt riêng biệt Nguồn Pi và Nguồn Motor
19	**Đầu giắc cắm nguồn Pi 4**	Giắc USB Type-C đực (bo 2 chân hàn)	1 cái	5.000đ	5.000đ	Hàn dây 5.15V cắm cổng Type-C của Pi 4
20	**Đốc sạc pin 18650 rời**	Đốc sạc đôi pin Li-ion cắm điện 220V	1 cái	28.000đ	28.000đ	Sạc rời pin độc lập, an toàn phòng chống cháy nổ
21	**Dây nối, ốc trụ đồng M3**	Dây bus Dupont cái-cái + Trụ đồng M3	1 gói	30.000đ	30.000đ	Đấu nối tín hiệu, bắt cố định bo mạch
TỔNG	**TỔNG CHI PHÍ**				**597.000đ**	Dự toán toàn diện, không phát

STT	Tên linh kiện	Model / Thông số kỹ thuật	SL	Đơn giá thực tế	Thành tiền	Ghi chú kỹ thuật
	MUA MỚI THỰC TẾ (CỘNG CHÍNH XÁC 100%):					sinh chi phí ẩn

(Kiểm toán phép tính số học: $48k + 16k + 54k + 28k + 115k + 32k + 5k + 4k + 10k + 160k + 12k + 10k + 28k + 6k + 6k + 5k + 28k + 30k = \mathbf{597.000đ}$. Toàn bộ tổng ngân sách dưới 600.000đ, hoàn toàn nằm trong khả năng tài chính của sinh viên).

PHẦN 5: SƠ ĐỒ ĐẤU DÂY TOÀN DIỆN & PHÂN BỔ CHÂN (PINOUT)

1. Bảng Phân Bổ Chân trên Vi Điều Khiển STM32F103C8T6

Ngoại vi kết nối	Chân trên Module	Chân STM32	Cấu hình Ngoại vi / Ghi chú Kỹ thuật
Nguồn cấp cho STM32	Ngõ ra 5.15V XL4015	**Chân 5V**	Nhận 5.15V từ Nhánh A, LDO onboard hạ 3.3V nuôi MCU
	Mass chung hệ thống	**GND**	Nối chung mass toàn hệ thống
Driver Động cơ L298N	`ENA`	**PA0**	Timer 2 Channel 1 (Hardware PWM) - Điều tốc bánh Trái
	`IN1` , `IN2`	**PA1**, **PA2**	GPIO Output Push-Pull - Đảo chiều quay cụm bánh Trái
	`ENB`	**PA3**	Timer 2 Channel 4 (Hardware PWM) - Điều tốc bánh Phải
	`IN3` , `IN4`	**PA4**, **PA5**	GPIO Output Push-Pull - Đảo chiều quay cụm bánh Phải
	`GND`	**GND**	Nối chung mass hệ thống
Cảm biến Đĩa quang (Encoder)	**D0 (Trái)**	**PB6**	EXTI Line 6 (Ngắt ngoài sườn lên đếm xung bánh Trái)
(2 Module LM393)	**D0 (Phải)**	**PB7**	EXTI Line 7 (Ngắt ngoài sườn lên đếm xung bánh Phải)

Ngoại vi kết nối	Chân trên Module	Chân STM32	Cấu hình Ngoại vi / Ghi chú Kỹ thuật
	`VCC` / `GND`	**3.3V / GND**	Cấp nguồn nuôi cảm biến từ STM32 (kèm trở kéo pull-up)
Giám sát Điện áp Pin 3S	Chân giữa Cầu phân áp	**PA6**	ADC1 Channel 6 (Đọc điện áp pin cảnh báo Soft-Shutdown)
Còi Báo động (Active Buzzer)	Cực Base Transistor	**PB5**	GPIO Output (Qua trở $1\text{k}\Omega$ kích mở 2N2222 đóng mass)
3 Cảm biến Siêu âm HC-SR04	`Trig` (Trái)	**PB12**	GPIO Output Push-Pull (Kích xung phát siêu âm Trái)
(Chân Echo chịu áp 5V an toàn)	`Echo` (Trái)	**PB13**	GPIO Input Floating (**Chân 5V-Tolerant (FT)**)
	`Trig` (Giữa)	**PB14**	GPIO Output Push-Pull (Kích xung phát siêu âm Giữa)
	`Echo` (Giữa)	**PB15**	GPIO Input Floating (**Chân 5V-Tolerant (FT)**)
	`Trig` (Phải)	**PB8**	GPIO Output Push-Pull (Kích xung phát siêu âm Phải)
	`Echo` (Phải)	**PB9**	GPIO Input Floating (**Chân 5V-Tolerant (FT)**)
Giao tiếp UART sang Pi 4	`TX1`	**PA9**	USART1_TX (Nối vào RX của mạch nạp USB-TTL)
	`RX1`	**PA10**	USART1_RX (Nối vào TX của mạch nạp USB-TTL)

---

2. Bảng Phân Bổ Kết Nối trên Raspberry Pi 4 Model B

Thiết bị kết nối	Cổng / Chân Thiết bị	Vị trí trên Pi 4	Chức năng Kỹ thuật
Webcam Logitech C270	Cáp đực USB 2.0	Cổng USB 2.0 (Đen)	Truyền luồng video 640×480 qua chuẩn UVC Video4Linux
Module USB-TTL CP2102	Cổng USB 2.0	Cổng USB 2.0 (Đen)	Giao tiếp Serial `/dev/ttyUSB0` tốc độ 115200 baud với STM32 (Tự lấy nguồn từ USB Pi)
Màn hình OLED 0.91" I2C	`VCC`	**Pin 1 (3.3V)**	Nguồn nuôi OLED từ thanh bus 3.3V của Pi
	`GND`	**Pin 9 (GND)**	Nối mass tín hiệu

Thiết bị kết nối	Cổng / Chân Thiết bị	Vị trí trên Pi 4	Chức năng Kỹ thuật
	`SDA`	**Pin 3 (GPIO 2)**	Tuyến dữ liệu I2C Bus 1 (Độc lập, không xung đột HAT)
	`SCL`	**Pin 5 (GPIO 3)**	Xung nhịp I2C Bus 1
Nguồn cấp chính hệ thống	Cáp tự chế USB Type-C	Cổng USB Type-C Pi	Nhận điện áp 5.15V ổn định từ module Buck XL4015 (qua Polyfuse 3A + TVS 5V)

PHẦN 6: CHIẾN LƯỢC ĐO ĐẠC THỰC NGHIỆM ĐỂ BẢO VỆ TRƯỚC HỘI ĐỒNG

Để đề tài đạt điểm số xuất sắc trước Hội đồng đánh giá và PGS. TS. Phan Văn Ca, nhóm không trình bày dưới dạng lý thuyết thuần túy mà xây dựng bộ dữ liệu kiểm chứng định lượng bằng thực nghiệm:

1. **Khảo sát Vùng chết Động cơ (Motor Deadband):**
2. Tăng dần duty cycle xung PWM từ 0% đến 60% trên từng bên bánh xe khi xe đặt trên mặt sàn gạch thực tế của lab.
3. Ghi nhận giá trị PWM tối thiểu (PWM_{min}) bắt đầu làm quay bánh xe và lập đồ thị quan hệ giữa PWM và vận tốc bánh xe để loại trừ hoàn toàn phi tuyến vùng chết trong bộ điều khiển PD.
4. **Đo đạc Dòng tiêu thụ Thực tế của Motor:**
5. Dùng ampe kế mắc nối tiếp kiểm tra dòng tiêu thụ ở 3 chế độ: Chạy không tải nhấc bánh, Chạy có tải tiến thẳng trên sàn, và Giữ chặt bánh (Stall).
6. Chứng minh bằng số liệu thực tế rằng dòng làm việc bình thường chỉ đạt $\approx 0.7 - 0.9\text{A}/\text{kênh}$ (nằm hoàn toàn trong ngưỡng an toàn 2A của L298N).
7. **Đồ thị Thực nghiệm Đáp ứng Bộ Điều Khiển P vs PD:**
8. Thu thập log thời gian thực của sai số lệch tâm $e_{norm}(t)$ qua file CSV trong quá trình robot di chuyển từ cự ly 2.0m về trạm.
9. Vẽ biểu đồ so sánh: trường hợp chỉ dùng bộ điều khiển P thuần túy (gây dao động lắc lư mũi xe) đối chiếu với trường hợp sử dụng bộ điều khiển PD có bộ lọc thông thấp (triệt tiêu dao động, đáp ứng tắt dần êm ái).
10. **Bảng Hiệu chuẩn Đo Khoảng cách ArUco (Camera Calibration Curve):**
11. Đặt robot tại các vị trí mốc cố định: $30\text{cm}, 50\text{cm}, 70\text{cm}, 100\text{cm}, 150\text{cm}$ (đo bằng thước lá chuẩn).
12. Ghi nhận kích thước pixel W_{px} đo được từ OpenCV, so sánh với công thức lý thuyết pinhole và tính toán sai số ước lượng khoảng cách.
13. **Thống kê Độ chính xác Cập trạm (Docking Accuracy) & Tỷ lệ Thành công:**
14. Cho robot thực hiện tối thiểu 30 lượt vận chuyển liên tục giữa các trạm (10 lượt mỗi cặp trạm $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$).

15. Dùng thước kẹp và thước lá đo sai số khoảng cách dừng ($\pm \text{mm}$) và sai số lệch tâm ngang ($\pm \text{mm}$) tại vị trí cập bến.
16. Tính tỷ lệ % hoàn thành nhiệm vụ thành công và phân tích các trường hợp thất bại (do ánh sáng lóa hoặc góc tới xiên của cảm biến).
17. **Benchmark Độ trễ Vòng Lặp Hệ thống & Tốc độ Khung hình (FPS):**
18. Sử dụng hàm `time.perf_counter()` trong Python để đo chi tiết thời gian thực thi của từng thành phần trong 1 chu kỳ:
    - *Capture frame C270 (tối ưu `cv2.CAP_V4L2` + FourCC `MJPG`):* $\approx 15 - 25\text{ms}$
    - *Detect ArUco & tính tọa độ:* $\approx 20 - 30\text{ms}$
    - *Tính toán PD & đóng gói UART:* $\approx 1 - 2\text{ms}$
    - *Tổng thời gian 1 chu kỳ điều khiển:* $\approx 40 - 65\text{ms}$ (tương ứng tốc độ khung hình thực tế kỳ vọng **$15 - 25\text{ FPS}$**, thông số chính xác sẽ được đo đạc và báo cáo cụ thể trong buổi bảo vệ).